DevOps — Herramientas y versiones

> Documento operativo del replatforming `ATPPuestos` (GeneXus 18 → Java 17 / Spring Boot + Thymeleaf). Última revisión: andamiaje del monorepo y migración a Thymeleaf.

---

1. Resumen

El despliegue es un **monolito modular**: **un solo artefacto desplegable** (fat JAR), **un solo proceso** y **una sola base de datos** (SQL Server). No hay microservicios, ni Spring Cloud, ni Eureka, ni API gateway, ni `docker-compose` multi-servicio. El módulo desplegable `atp-puestos-camineros` absorbe al núcleo (`atp-nucleo-compartido`) en el JAR final.

2. Matriz de herramientas y versiones

2.1 Runtime y build (obligatorios)

Herramienta	Versión	Fuente / nota
JDK	**17 (LTS)**	`java.version=17` en el parent POM. `JAVA_HOME=C:\Program Files\Java\jdk-17`.
Spring Boot (parent POM)	**3.5.16**	`spring-boot-starter-parent` en `pom.xml` raíz.
Maven Wrapper	(incluido en el repo)	`./mvnw` / `mvnw.cmd`. No se requiere Maven global.
Build	Maven multi-módulo	Módulos: `atp-nucleo-compartido`, `atp-puestos-camineros`.
Empaquetado	`spring-boot-maven-plugin` (fat JAR)	`atp-puestos-camineros` depende del núcleo ⇒ un JAR con todo adentro.

2.2 Persistencia y base de datos

Herramienta	Versión / valor	Fuente / nota
Motor BD	SQL Server (existente)	
Driver JDBC	`mssql-jdbc 12.10.2`	Verificado en `application.yml` (arranque real + suite IT).
ORM	Spring Data JPA / Hibernate 6.6	`hibernate.hbm2ddl.auto=none` (obligatorio).
Dialecto	`org.hibernate.dialect.SQLServer2012Dialect`	Evita el aviso HHH000511 (verificado en arranque real).
Pool de conexiones	HikariCP (`atp-puestos-pool`)	`maximum-pool-size=10`, `connection-timeout=10000`.
Migraciones	**Ninguna**	Prohibido Flyway, Liquibase, `schema.sql`/`data.sql` y DDL.

2.3 Presentación y web

Herramienta	Versión / valor	Fuente / nota
Motor de plantillas	Thymeleaf	Prefijo `classpath:/templates/`.
Web framework	Spring MVC (`@Controller` + `@RestController`)	Dos canales: `/api/**` JSON y el resto HTML.
Estilos	CSS propio con custom properties (tokens)	Derivado de `ATPEvo2.css`. Sin framework, sin CDN, sin build.
JavaScript	Vanilla (puntual)	`*/src/main/resources/static/js/`. Sin `npm`, bundler ni transpilador.
Seguridad	Spring Security	Sesión HTTP + CSRF. Cookie `SameSite=Lax`, `HttpOnly`.

2.4 Pruebas

Herramienta	Cuándo corre	Fuente / nota
JUnit 5 + Mockito	siempre (`mvn test`)	Unitarios.

Herramienta	Cuándo corre	Fuente / nota
`@WebMvcTest` + MockMvc	siempre	Render, binding, redirección, contrato REST.
Paridad SQL	con evidencia	Comparar consultas Hibernate vs. *Navigation Report* de GeneXus.

2.5 Análisis de la KB legacy (sólo desarrollo, no runtime)

Herramienta	Versión	Fuente / nota
GeneXus	18.0.13 (U13)	`C:\Program Files (x86)\GeneXus\GeneXus18`. Instalación con licencia válida (el SDK la requiere aunque no se abra el IDE).
Node.js	(disponible en PATH)	Requerido para `npx`/`npx.cmd`.
`genexus-mcp`	**3.9.2** (fijado, no `@latest`)	Servidor MCP sobre la KB. En Windows el ejecutable debe ser `npx.cmd`.
`GX_CONFIG_PATH`	`F:\Modelo\Gx18\ATPPuestos\config.json`	Archivo `config.json` **de la carpeta de la KB** (no el de la instalación de GeneXus).

Estas herramientas son de **introspección en modo lectura** sobre la KB legacy.
No forman parte del runtime ni del artefacto desplegable.

3. Variables de entorno

Plantilla: `.env.example` (nunca versionar `.env`). Regla G3: sin secretos en el repositorio.

Variable	Propósito
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_DRIVER`	Conexión a SQL Server
`SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, `SPRING_DATASOURCE_PASSWORD`	Alternativa completa a las anteriores.

Variable	Propósito
`SPRING_PROFILES_ACTIVE`	Perfil de Spring (`dev` por defecto).
`SERVER_PORT`	Puerto del servidor (`8080` por defecto).
`APP_BASE_URL`	URL pública de la app (enlaces absolutos y redirects post-login).
`SECURITY_SESSION_TIMEOUT_MINUTES`	Timeout de sesión (`30`).
`SECURITY_COOKIE_SECURE`, `SECURITY_COOKIE_HTTPONLY`, `SECURITY_COOKIE_SAMESITE`	Flags de la cookie de sesión.
`APP_AUDITORIA_USUARIO`	Usuario auditor por defecto sin sesión (`SISTEMA`).

4. Build y pruebas

```shell
# JDK 17 obligatorio
set JAVA_HOME=C:\Program Files\Java\jdk-17

# Copiar plantilla y completar credenciales
copy .env.example .env

# Build + tests unitarios + vistas + contrato REST
.\mvnw.cmd -B clean test

# Integración contra la BD real (gated)
set ATP_DB_IT=true
.\mvnw.cmd -B test "-Dtest=*IT" "-Dsurefire.failIfNoSpecifiedTests=false"
```

En PowerShell los argumentos de Maven con punto van **entrecomillados**; sin comillas, `-Dsurefire.x` se parte y Maven lo interpreta como una fase del ciclo de vida.

5. Ejecución

```shell
# Levantar la app
.\mvnw.cmd -pl atp-puestos-camineros spring-boot:run
```

- Puerto efectivo: `${SERVER_PORT:8080}`.
- Perfil efectivo: `${SPRING_PROFILES_ACTIVE:dev}`.

- Configuración: `application.yml` vive en el **núcleo** (`atp-nucleo-compartido/src/main/resources/`). Es la única fuente de verdad: Spring Boot toma el primer `classpath:/application.yml` y **no fusiona** varios. Lo específico de un módulo va en `application-<perfil>.yml` propio de ese módulo.
- `ddl-auto=none`: Hibernate **nunca** genera DDL sobre el esquema existente.
- `open-in-view=false`.

6. Despliegue (estado actual)

- Artefacto único: fat JAR del módulo `atp-puestos-camineros` (incluye el núcleo).
- Un solo proceso, una sola base .
- Sin contenedores, sin orquestador, sin pipeline CI/CD versionado en el repo.

7. Pendientes / huecos (fuera de alcance actual)

No existen en el repositorio, y se documentan como pendientes para una futura fase de operaciones:

- `Dockerfile` / imagen de contenedor.
- Pipeline CI/CD (build, test, publicación del artefacto).
- Manifiesto de despliegue (systemd, servicio Windows, etc.).
- Monitoreo/observabilidad en producción (logs, métricas, alertas).

Checklist mínima para poner en producción (cuando se decida):

1. JDK 17 instalado en el host y `JAVA_HOME` apuntando a él.
2. Variables de entorno cargadas (sección 3), con `SECURITY_COOKIE_SECURE=true` y perfil `prod`.
3. Base `Rentas` accesible desde el host.
4. Puerto `SERVER_PORT` libre y expuesto.
5. Artefacto fat JAR generado con.`\mvnw.cmd -B clean package` y ejecutado con `java -jar`.

Fuentes: `pom.xml`, `atp-nucleo-compartido/src/main/resources/application.yml`, `.env.example`, `openspec/constitution/stack.md`, `openspec/constitution/conventions.md`.